Reporte

Nombre: Andrés Daniel Pérez Ordaz Jonathan Esquivel Anaya Patricia Lizeth Ramos Ortiz	**Matrícula:** 2163200 2160886 2161739
Nombre del curso: Componentes de lógica de negocio	**Nombre del profesor:** Miguel Ángel Gómez Marroquín
Modulo Netacad: 3	**Actividad:** Actividad Unidad 3: Diseño arquitectural del sistema con IA Bitácora de IA
Fecha: 17/10/2026	

Bibliografia (Formato APA)

Evans, E. (2017). Domain-Driven Design: Tackling Complexity in the Heart of Software (Trad. al español). Addison-Wesley / Pearson Educación.

Freund, J., & Rücker, B. (2020). BPMN 2.0: Manual de referencia y guía práctica. Marcombo.

Bitácora de IA

Esta sección documenta cómo se usó la IA como copiloto en el diseño arquitectural de CleanPro. Se ejecutaron los cinco prompts de apoyo de la actividad (patrones, dominio, Service Layer, SOLID y diagramas) y una sexta iteración de revisión final de coherencia. Cada uno se completó con información de la Unidad 2 (contexto del caso, BRD, historias de usuario con Gherkin, glosario y BPMN) y cada respuesta se contrastó con el diseño del equipo. Cada iteración sigue el ciclo de la actividad: preparar contexto, solicitar propuesta, comparar, corregir, validar y registrar.

Fuentes de contexto usadas en todas las iteraciones:

CleanPro (caso 12), BRD, User Stories US-01 a US-19, Glosario y BPMN. Nomenclatura de reglas: BR-01 cotización sin cuadrilla definitiva; BR-02 confirmación un día antes; BR-03 ausencias; BR-04 extras y catálogo; BR-05 ajuste por inmueble mayor; BR-06 reprogramación; BR-07 cierre para facturación; BR-08 modificación o cancelación. RNF-01 visibilidad de cambios; RNF-02 permisos por rol; RNF-03 historial de cambios; RNF-04 políticas configurables.

Resumen de la bitácora

Iteración	Objetivo	Prompt	Respuesta útil	Error o supuesto	Decisión del equipo

1	Comparar patrones de lógica de negocio	Patrones (completado con BR-01 a BR-08, políticas pendientes y CU-01 a CU-05)	Tabla comparativa con consecuencias de Transaction Script, Table Module y Domain Model, y condiciones bajo las que cada uno conviene	Supuestos declarados: equipo pequeño con tiempo para modelar y políticas que cambiarán al validarse. Se verificó que la justificación se apoyara en evidencia del caso y no en argumentos generales como «escalabilidad»	Domain Model para el flujo que cambia estado; Transaction Script para lecturas (ADR-001)
2	Proponer elementos del dominio	Dominio (glosario U2 + reglas BR/US)	Candidatos a entidades, Value Objects, Aggregates y eventos, cada uno con evidencia y dudas	Factura, Cliente y el estado Suspendida se señalaron como candidatos dudosos, sin evidencia suficiente en la U2. El folio único y la definición de Empleado quedaron como dudas	Se descartan Factura, Cliente y Suspendida; folio único como supuesto pendiente; Empleado y Cuadrilla como raíces mínimas
3	Firmas de Application Services	Service Layer (CU-01 a CU-05)	Cinco servicios con firma, colaboradores y reglas delegadas al dominio	Dos servicios de aplicación tentaban a decidir (permiso por rol y «precio significativo»). Faltaba EmpleadoRepository para reasignar	Las decisiones quedan en el Aggregate Servicio y en PoliticaDeExtras; se agrega EmpleadoRepository
4	Detectar violaciones SOLID	SOLID (diseño completo de Etapas 3 a 6)	Seis hallazgos con evidencia, riesgo y alternativas	Algunos riesgos son de crecimiento futuro, no de un defecto actual. La IA no puede decidir cuáles pesan más	Se corrigen RepositoriosSql, la mezcla Cotización/Servicio en una transacción y la reformulación de LSP; el resto queda como riesgo documentado
5	Borrador de diagramas	Diagramas (arquitectura validada → Mermaid)	Código Mermaid del diagrama de clases y de dos de secuencia, para pegar en draw.io	La primera versión tenía unas 45 clases y participantes que no estaban en el	Se usa la versión reducida y los participantes de las secuencias se alinean con el

				diagrama de clases. Se redujo a 22 clases	diagrama de clases
6	Revisar coherencia, concordancia y continuidad de todos los entregables	Revisión final (documentos, diagramas y código contra las instrucciones y la U2)	Lista de discrepancias y correcciones directas con su justificación	Códigos de hallazgo sin definir, contradicción entre SOLID y bitácora, tablas distintas a los diagramas y elementos de diseño presentados como implementados	Se corrigen informe, bitácora, glosario y matriz; se agrega prueba de contrato al código; se actualizan los fuentes de los diagramas

Iteración 1 · Patrones de lógica de negocio

Objetivo: comparar Transaction Script, Table Module y Domain Model con base en el comportamiento real de CleanPro, antes de elegir.

Contexto entregado

- Reglas BR-01 a BR-08 del BRD y RNF-01 a RNF-04.
- Cambios esperados: tiempo de cortesía, cobro por acceso negado, penalización por reprogramación tardía, catálogo de extras y criterio de precio «significativo». Todos pendientes de validar con el negocio.
- Casos de uso CU-01 a CU-05: aceptar cotización, confirmar cuadrilla, reportar ausencia y reasignar, registrar extra en sitio y cerrar para facturación.

Prompt

Con base en estas reglas (BR-01 a BR-08), estos cambios esperados (tiempo de cortesía, cobro por acceso negado, penalización por reprogramación tardía, catálogo de extras y criterio de precio significativo, todos pendientes de validar) y estos casos de uso (CU-01 aceptar cotización, CU-02 confirmar cuadrilla el día previo, CU-03 reportar ausencia y reasignar, CU-04 registrar extra en sitio, CU-05 cerrar para facturación), compara Transaction Script, Table Module y Domain Model. Declara tus supuestos y no elijas una opción hasta mostrar sus consecuencias.

Respuesta de la IA (resumen)

Supuestos declarados:

- El sistema lo mantiene un equipo pequeño, con tiempo para modelar el dominio.
- Las políticas pendientes cambiarán cuando el negocio las valide (RNF-04).
- No hay requisito de rendimiento extremo ni integración contable (BRD).
- Las consultas (disponibilidad, panel, historial) no protegen invariantes.

Criterio	Transaction Script	Table Module	Domain Model

Reglas que cruzan conceptos (BR-03, BR-07)	Cada procedimiento repite la verificación de pendientes antes de cerrar	La regla queda repartida por tabla (Servicio, Incidencia, Extra); alguien tiene que coordinarla	La regla vive en Servicio, que contiene incidencias y extras
Estados e invariantes (S1 a S13)	El estado es un valor que cada script valida	El estado es una columna validada por módulo	Las transiciones están dentro del Aggregate; no hay otro camino a un estado inválido
Políticas que cambian	Hay que editar cada script que las use	El cambio queda en el módulo, pero sin objeto que represente la política	Se encapsulan en un objeto de configuración y no tocan el flujo
Costo inicial	Bajo	Bajo a medio	Alto: modelado y mapeo a persistencia
Riesgo principal	Duplicar BR-07	Terminar como Transaction Script con otro nombre	Sobre-modelar (una entidad por tabla)

Consecuencias que señaló antes de concluir: Transaction Script es suficiente si el sistema se limitara a registrar eventos sin proteger estados; Table Module sirve para reportes por tabla; Domain Model solo compensa si las invariantes entre Servicio, Incidencia y Extra son centrales (BR-07, BR-02, US-08, US-11). Conclusión condicionada: Domain Model para el flujo que cambia estado y Transaction Script para lecturas.

Revisión y corrección

- Los argumentos genéricos («escalable», «moderno», «más mantenible») no sirven: la actividad pide evidencia del caso. La justificación se ancló en BR-07, BR-02, US-08 y US-11.
- El tamaño del sistema (19 historias, 5 actores) no justifica por sí solo el patrón; la justificación es la cantidad de estados e invariantes.
- Se verificó cada afirmación contra el BRD y las historias. Las referencias a BR-xx coinciden con el BRD.

Decisión del equipo

Domain Model para el flujo que cambia el estado del Servicio; Transaction Script solo para consultas; Table Module descartado. Queda registrado en ADR-001.

Iteración 2 · Mapa de dominio

Objetivo: obtener candidatos a entidades, Value Objects, Aggregates y Domain Events, cada uno con su evidencia y con las dudas que requieren validación humana.

Contexto entregado

- Glosario completo de la U2 (23 términos).
- Reglas BR-01 a BR-08 y las historias US-01 a US-19.

- BPMN: carriles Cliente, Coordinador, Cuadrilla, Supervisor y Administración, y sus puntos de decisión.

Prompt

Revisa este glosario y estas reglas [glosario de CleanPro: Cotización, Programación tentativa, Confirmación de cuadrilla, Servicio, Cuadrilla, Ausencia, Reasignación de personal, Servicio en riesgo, Incidencia, Extra, Catálogo de extras, Ajuste de precio y tiempo, Tiempo de cortesía, Servicio no realizado, Material especial, Reprogramación, Cancelación, Registro de inicio y término, Cierre para facturación, Panel operativo, Historial de servicio; reglas BR-01 a BR-08]. Propón candidatos a entidades, Value Objects, Aggregates y Domain Events. Para cada candidato, indica la evidencia utilizada y las dudas que requieren validación humana.

Respuesta de la IA (resumen)

Tipo	Candidatos	Evidencia	Duda para validar
Aggregate / Entidad	Cotización (Folio), Servicio (Folio), Cuadrilla, Empleado, ConfiguracionDePoliticas	Glosario: cotización con folio y estados; Servicio agrupa cotización, cuadrilla y registros; BR-01, BR-02	¿El Servicio conserva el folio de la cotización o tiene uno propio? (US-03 vs US-06 y US-08)
Entidad dentro de Servicio	Incidencia, Extra, AjusteDePrecioYTiempo, SolicitudDeReprogramacion	US-09, US-10, US-11, US-14 y US-15: cada uno tiene estado y ciclo propio	Qué pasa si el cliente rechaza un ajuste o el coordinador rechaza un extra (no definido)
Value Object	Folio, DatosDelInmueble, Frecuencia, Estimacion, Dinero, RegistroDeTiempo, AsignacionDePersonal, FechaDeServicio, Motivo	US-01, US-02, US-07, US-08, RNF-03	Moneda del Dinero no definida
Domain Service	CalculadoraDeEstimacion, AsignadorDeCuadrilla, PoliticaDeExtras	US-02, US-04, US-05, US-10	El tarifario no está definido en la U2
Domain Events	CotizacionEmitida, CotizacionAceptada, ServicioProgramado, CuadrillaConfirmada, AusenciaReportada, PersonalReasignado, ServicioEnRiesgo, IncidenciaDocumentada, ExtraAceptado, ServicioCerradoParaFacturacion, entre otros	Historias US-03 a US-17; eventos finales del BPMN	Qué eventos requieren notificación inmediata además de AusenciaReportada (RNF-01)
Candidatos dudosos	Factura, Cliente, estado Suspendida	Aparecen en el BPMN («Facturar servicio»), en el caso (Cliente como actor) y en BR-08 (modificar o cancelar)	Sin evidencia de comportamiento propio: BRD excluye facturación externa; Cliente solo solicita y acepta; BR-08

			solo habla de modificar o cancelar

Revisión y corrección

- Factura: se descartó. El BRD deja fuera la integración con sistemas contables y el sistema solo cierra para facturación. El cierre emite el evento ServicioCerradoParaFacturacion.
- Cliente: se descartó como Aggregate; solo se referencia por ClienteId. La U2 no define datos ni comportamiento propios del cliente.
- Suspendida: se descartó. No existe en el glosario ni en las historias.
- Se verificó que ninguna respuesta introdujera reglas o cifras nuevas (rangos de superficie, descuentos, límite de cuadrillas): la U2 no las define y quedan como supuestos pendientes.
- Empleado no está definido en el glosario; se modeló como raíz mínima (EmpleadoId).

Decisión del equipo

Cotización y Servicio como Aggregates separados con el mismo Folio (supuesto pendiente de validar), Cuadrilla y Empleado como raíces mínimas, ConfiguracionDePoliticas como soporte, y los elementos descartados registrados en «Decisiones dudosas o descartadas». Se refleja en ADR-002 y ADR-003.

Iteración 3 · Service Layer

Objetivo: obtener firmas de Application Services que coordinen cada caso de uso sin absorber las reglas del dominio.

Contexto entregado

- CU-01 a CU-05 con actor, reglas y resultado esperado.
- Invariantes S1 a S13 del Aggregate Servicio y las reglas BR-01 a BR-08.

Prompt

A partir de estos casos de uso [CU-01 aceptar cotización (Cliente), CU-02 confirmar cuadrilla el día previo (Coordinador), CU-03 reportar ausencia y reasignar personal (Supervisor y Administración), CU-04 registrar extra solicitado en sitio (Supervisor), CU-05 cerrar servicio para facturación (Administración)], propone firmas de Application Services. Separa coordinación de reglas de dominio y explica cualquier responsabilidad dudosa.

Respuesta de la IA (resumen)

Servicio	Método	Coordina	Delega al dominio

CotizacionesAppService	aceptarCotizacion(folio, clienteId)	Obtiene la Cotización, la acepta, crea el Servicio programado y confirma la transacción	Cotización: solo una cotización Vigente se acepta. Servicio: nace en estado Programado sin cuadrilla (BR-01)
ProgramacionAppService	confirmarCuadrilla(folio, cuadrillaId, coordinadorId)	Obtiene Servicio y Cuadrilla, consulta AsignadorDeCuadrilla, guarda y confirma	Servicio: transición a CuadrillaConfirmada solo si hay disponibilidad (BR-02)
PersonalAppService	reportarAusencia(folio, empleadoId, supervisorId); reasignarPersonal(folio, empleadoId, sustitutoId, adminId)	Obtiene Servicio y Empleado, guarda, confirma y avisa por NotificacionPort	Servicio: crea la incidencia, conserva original y sustituto, pasa a EnRiesgo sin reemplazo (BR-03)
EjecucionAppService	registrarExtra(folio, extra, supervisorId)	Obtiene Servicio y políticas vigentes, guarda y confirma	PoliticaDeExtras decide si el extra está en el catálogo; Servicio lo registra aceptado o pendiente (BR-04)
CierreAppService	cerrarParaFacturacion(folio, adminId)	Obtiene Servicio, guarda, confirma y publica el evento por FacturacionPort	Servicio: bloquea el cierre con pendientes y señala cuál falta (BR-07)

Responsabilidades dudosas señaladas por la IA:

- Verificar que quien cierra para facturación sea Administración (RNF-02): puede estar en el servicio de aplicación como permiso de acceso, o en el dominio como regla del actor.
- Decidir cuándo un extra «modifica el precio de forma significativa»: el criterio no está definido y no debe quedar en el servicio.
- aceptarCotizacion toca dos Aggregates (Cotización y Servicio) en una sola operación.
- Para reasignar personal hace falta consultar empleados disponibles, lo que requiere un repositorio de empleados.

Revisión y corrección

- Se agregó EmpleadoRepository, que no estaba en la primera tabla de Repository.
- El criterio de «precio significativo» se dejó en ConfiguracionDePoliticas y no en el servicio.
- Quién puede ejecutar cada operación se pasa al dominio como actor con rol, para que la regla del cierre (solo Administración) viva con S12 y no en la capa de aplicación.

- Para aceptarCotizacion se mantuvo la excepción documentada de dos Aggregates en una operación; queda como riesgo (ver iteración 4).

Decisión del equipo

Se conservan los cinco servicios con estas firmas. Los servicios coordinan; los umbrales, los permisos por rol y las transiciones viven en el dominio.

Iteración 4 · Revisión SOLID

Objetivo: detectar posibles violaciones SOLID del diseño con evidencia, riesgo y alternativas, sin reescribir todo.

Contexto entregado

- Mapa de dominio (Aggregates, Domain Services, eventos).
- Service Layer, interfaces de Repository, Unit of Work y CQRS básico.
- Diagrama de clases reducido de 22 clases.

Prompt

Analiza este diseño [Aggregates Servicio, Cotización, Cuadrilla, Empleado y ConfiguracionDePoliticas; Domain Services AsignadorDeCuadrilla y PoliticaDeExtras; cinco Application Services con Unit of Work; cinco interfaces de Repository implementadas por RepositoriosSql; puertos NotificacionPort y FacturacionPort] y señala posibles violaciones SOLID. No reescribas todo; identifica evidencia, riesgo y alternativas para que el equipo decida.

Respuesta de la IA (resumen)

Principio	Evidencia	Riesgo	Alternativas
SRP	Servicio concentra 13 invariantes (S1 a S13) y 8 estados	Aggregate demasiado grande: cada política nueva lo modifica	Extraer objetos de estado o políticas; dividir si crece, sin hacerlo todavía
SRP / ISP	RepositoriosSql implementa las cinco interfaces de Repository	Una clase con varias razones para cambiar y con métodos que algunos clientes no usan	Una implementación por Repository (SqlServicioRepository, etc.); la caja única solo simplifica el diagrama
ISP	ServicioRepository incluye obtenerConfirmadosPorFecha, que solo usa AsignadorDeCuadrilla	Interfaz que crece con consultas	Separar una interfaz de consulta de disponibilidad
OCP	PoliticaDeExtras recibe la configuración por parámetro	Bajo mientras el catálogo sea un dato; si aparecen reglas distintas por tipo de extra habrá condicionales	Mantener como configuración; pasar a estrategias si las reglas cambian de forma, no solo de valor

LSP	Los repositorios devuelven Aggregates completos y se sustituyen por una versión en memoria en pruebas	Bajo, si los contratos no cambian entre implementaciones	Pruebas de contrato que ejecuten la misma batería sobre ambas implementaciones
DIP	AsignadorDeCuadrilla (dominio) depende de ServicioRepository; aceptarCotizacion usa dos Aggregates en una operación	Un Domain Service que consulta repositorios acopla el dominio a la persistencia; y la excepción de dos Aggregates rompe la regla de uno por transacción	Pasar la lista de servicios confirmados al Domain Service como dato; reaccionar al evento CotizacionAceptada para crear el Servicio

Revisión y corrección

- El hallazgo de SRP sobre Servicio es un riesgo de crecimiento, no un defecto actual. Se documentó y no se dividió el Aggregate, porque dividirlo rompería S12 (BR-07).
- RepositoriosSql es una simplificación del diagrama reducido. En el código y en el diagrama completo cada Repository tiene su propia implementación.
- El hallazgo 3 de la revisión original (LSP) hablaba de uso de interfaces, que es DIP. Se reformuló con el criterio de sustitución mediante pruebas de contrato.
- AsignadorDeCuadrilla consultando ServicioRepository se mantiene por ahora; se anota como compromiso: es una dependencia de lectura a una abstracción, no a SQL.

Decisión del equipo

Se corrigen la implementación única de repositorios (una por Repository) y la reformulación de LSP, que ahora se respalda con pruebas de contrato (agregadas en la iteración 6). aceptarCotizacion se mantiene como excepción documentada en ADR-002. Los demás hallazgos se registran como riesgos a vigilar. En el informe los seis hallazgos se numeran SOL-1 a SOL-6.

Iteración 5 · Diagramas

Objetivo: convertir la arquitectura ya validada en borradores Mermaid del diagrama de clases y de dos diagramas de secuencia, sin agregar clases, reglas ni relaciones nuevas.

Contexto entregado

- Descripción de la arquitectura validada: 5 Application Services, 8 elementos del dominio, 5 interfaces de Repository, Unit of Work, 2 puertos y los adaptadores de infraestructura.
- Secuencias a dibujar: confirmar cuadrilla (CU-02) y cerrar para facturación con bloqueo (CU-05).

Prompt

Convierte esta arquitectura validada [Application Services: CotizacionesAppService, ProgramacionAppService, PersonalAppService, EjecucionAppService, CierreAppService; dominio: Servicio, Cotizacion, Incidencia, Extra, AsignacionDePersonal, Cuadrilla, Empleado, ConfiguracionDePoliticas, AsignadorDeCuadrilla, PoliticaDeExtras; Repository: Servicio, Cotizacion, Cuadrilla, Empleado, Configuracion; UnitOfWork, NotificacionPort, FacturacionPort; infraestructura: RepositoriosSql, SqlUnitOfWork, NotificacionAdapter, FacturacionEventAdapter] en un borrador Mermaid para pegar en draw.io. No agregues clases, reglas o relaciones que no aparezcan en la descripción.

Respuesta de la IA (resumen)

- Un diagrama de clases con 22 clases, separando capas con estereotipos (<<Aplicacion>>, <<Aggregate Root>>, <<Infraestructura>>), relaciones de dependencia, composición, implementación y agregación.
- Una secuencia de flujo principal (confirmar cuadrilla) y una de flujo con regla (cierre bloqueado por BR-07), cuyos participantes son clases del diagrama de clases.
- El código se entrega como texto plano y se valida con el compilador de Mermaid antes de entregarlo.

Revisión y corrección

- La primera versión del diagrama de clases tenía unas 45 clases (cada Repository con su implementación SQL, DTO de lectura, puertos y todos los Value Objects). El enunciado pide las clases relevantes del proceso elegido: se redujo a 22.
- La primera versión de las secuencias incluía pasos y participantes que no estaban en el diagrama de clases (por ejemplo, un paso «iniciar operación» y anotaciones). Se alinearon con las clases y se quitaron.
- Para que las secuencias fueran consistentes se agregaron tres flechas al diagrama de clases: ProgramacionAppService hacia ServicioRepository y hacia UnitOfWork, y CierreAppService hacia ServicioRepository.
- En el fuente Mermaid las capas se agrupan con namespaces (Aplicacion, Dominio, Infraestructura) y cada clase lleva su estereotipo; no hace falta dibujar franjas a mano.
- En un primer intento el diagrama se generó con otra herramienta, con flechas cruzadas en todas direcciones; se descartó y se usó Mermaid.

Decisión del equipo

Se usan el diagrama de clases reducido y las dos secuencias alineadas. Al revisar el diagrama contra el código (iteración 6) quedó con 36 elementos: 11 de aplicación, 16 de dominio y 9 de infraestructura, porque incluye el lado de lectura (ConsultasAppService, DisponibilidadReadRepository, DisponibilidadDTO), las

entidades Incidencia y Extra y una implementación por Repository. Los archivos fuente Mermaid van en la carpeta diagrams del repositorio.

Iteración 6 · Revisión final de coherencia

Objetivo: revisar la coherencia, concordancia y continuidad del informe, la bitácora, el glosario, la matriz, los diagramas y el código contra las instrucciones de la actividad y los productos de la Unidad 2, y corregir cada documento justificando el cambio.

Contexto entregado

- Instrucciones de la actividad (etapas, entregables, rúbrica y lista de comprobación).
- Productos de la U2: BRD, User Stories con Gherkin, BPMN y Glosario.
- Documento original AF03 y los entregables derivados: informe, bitácora, glosario actualizado y matriz de trazabilidad.
- Fuentes Mermaid del diagrama de clases, del mapa de dominio y de las dos secuencias.
- Código base completo con sus pruebas (29 al inicio de la revisión).

Prompt

Revisa que los documentos concuerden con los diagramas, con la Unidad 2 y con las instrucciones de la actividad. Corrige directamente los documentos, justifica cada cambio y no inventes reglas ni valores que la U2 no define.

Respuesta de la IA (resumen)

Hallazgo	Evidencia	Corrección	Documento
Casos de uso sin tabla	El informe listaba CU-01 a CU-05 sin actor, reglas ni resultado, que la Etapa 1 exige; el AF03 original sí los tenía	Se restauró la tabla de casos de uso, la evidencia en la U2 y el flujo de excepción de cada uno	Informe
Códigos de hallazgo sin definir	HC-02, HC-03, HC-06, HC-10 y HC-11 aparecían en informe, diagramas y código sin ninguna tabla que los definiera	Se sustituyeron por una tabla única H-01 a H-08 y por referencias a ADR-002 y ADR-004	Informe, diagramas, código
Contradicción en SOLID	La bitácora hablaba de correcciones; el informe y las conclusiones decían que los cinco hallazgos solo confirmaban	Revisión rehecha con seis hallazgos (SOL-1 a SOL-6) con evidencia, impacto y cambio realizado; conclusiones ajustadas	Informe
LSP sin evidencia	Las pruebas de contrato no existían en el código y el hallazgo de ISP citaba un	Se agregó tests/test_contrato_repositorios.	Informe, código

	método eliminar() que no existe	py (6 pruebas, 35 en total) y se quitó eliminar()	
Tablas distintas a los diagramas	EjecucionAppService sin ConfiguracionRepository; faltaban confirmarExtra, revisarIncidencia, EmpleadoRepository y obtenerConfirmadosPorFecha; los ReadRepository tenían otros nombres	Tablas de Service Layer, Repository y CQRS alineadas con el diagrama de clases; PanelReadRepository marcado como propuesta	Informe
Diseño presentado como implementado	CalculadoraDeEstimacion, AjusteDePrecioYTiempo, SolicitudDeReprogramacion, S7 a S11 y varios eventos no están en el código (README: fuera del alcance)	Se marcaron como diseño sin implementar	Informe, mapa de dominio
Errores de texto	«Confirms cuadrilla»; documentarIncidencia() no existe en la capa de aplicación; «fecha de actualización» era un campo inventado; el ORM se presentaba como si estuviera implementado	Corregidos; se aclaró que el código usa repositorios en memoria y que el ORM es la propuesta de producción	Informe
Numeración sin respaldo	El BRD no numera sus reglas ni sus RNF; el cuarto RNF repite la regla de cierre	Se documentó la nomenclatura del equipo (H-07)	Informe, glosario, matriz
Bitácora desactualizada	Decía 24 términos en el glosario de la U2 (son 23), un diagrama de 22 clases y no reflejaba la revisión final	Corregido; se agregó esta iteración	Bitácora
Matriz incoherente	Asignaba PersonalAppService y CotizacionesAppService a ajustes y reprogramación, describía CU-04 como incidencias y control de acceso y mezclaba cierre con panel	Matriz rehecha con una fila por regla, estado en el código y hallazgos	Matriz
Glosario y ADR-002	El glosario decía que el Servicio agrupa la cotización, pero ADR-002 las separa en dos Aggregates	Se precisó que el Servicio la referencia por su Folio	Glosario
Diagramas incompletos	Sin flechas de los Application Services hacia Servicio ni de AsignadorDeCuadrilla hacia Cuadrilla; sin	Fuentes Mermaid actualizados	Diagramas

	CalculadoraDeEstimacion en el mapa de dominio		

Revisión y corrección

- Cada cambio se contrastó con el BRD, las User Stories, el BPMN y el glosario de la U2, y con el código ejecutado (las 35 pruebas pasan).
- No se agregaron reglas ni valores nuevos: lo que la U2 deja pendiente sigue como supuesto, política configurable o diseño sin implementar.
- Los diagramas ya exportados (PNG o draw.io) deben regenerarse con los fuentes Mermaid actualizados antes de entregar.

Decisión del equipo

Se aceptan las correcciones, se conserva la tabla H-01 a H-08 como lista única de hallazgos sobre la U2 y se regeneran los diagramas desde los fuentes actualizados.